

Project Documentation

Project-1

YOLO-based CCTV Idle Detection and Excel Logging System

1.Description:

The goal of this project is to create and put into action a real-time CCTV monitoring system that utilizes computer vision and machine learning to spot people, track their movements, and identify those who are idle. The system takes advantage of the YOLOv8 (You Only Look Once) deep learning model for detecting objects and OpenCV for processing video footage. Additionally, it connects with Excel to generate reports on idle activities for future analysis.

This project is designed to support various applications like:

- a. Monitoring productivity in the workplace
- b. Ensuring security in restricted areas
- c. Analyzing crowd behavior

2.System Objectives:

The system is designed with the following goals:

1. Real-time monitoring of CCTV streams using RTSP protocol.
2. Detection and tracking of people using YOLOv8.
3. Idle detection based on movement over time.
4. Data logging into an Excel report for auditing and analysis.
5. Visualization of active vs. idle people in the video feed

3.Tools and Technologies:

- **Programming Language:** Python idle 3.11
- **Libraries and Frameworks:**
 - OpenCV → Video capture and display
 - Ultralytics YOLOv8 → Object detection model
 - OpenPyXL → Excel logging and data management
 - Threading → multi-camera frame grabbing
- **Hardware Requirements:**
 - Computer with GPU (recommended for YOLOv8)

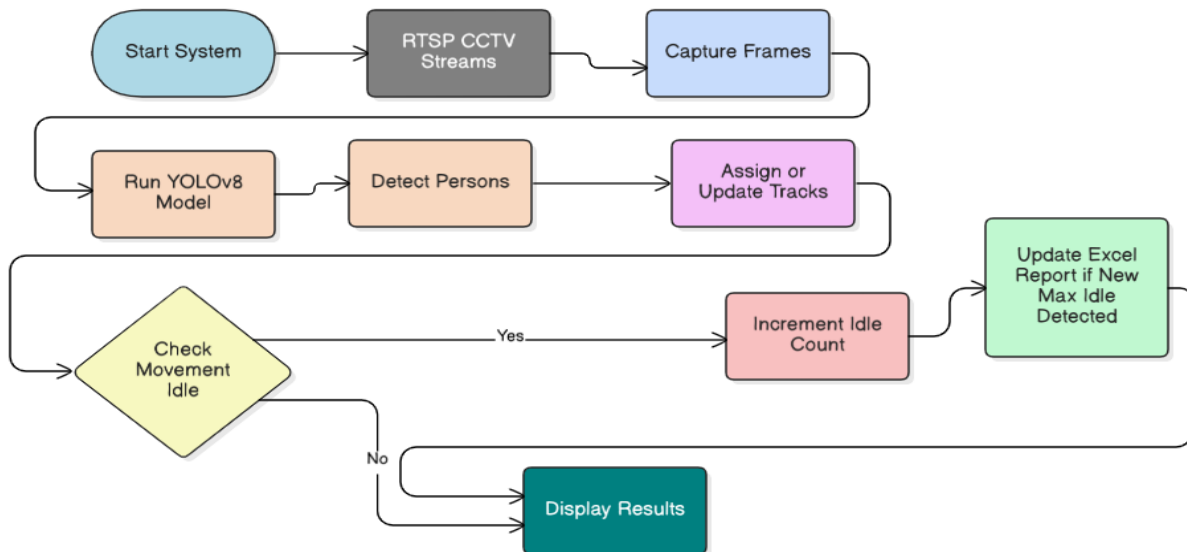
- CCTV Cameras with RTSP streaming support
- **Software Requirements:**
 - Python environment with dependencies installed
 - Excel viewer (MS Excel, LibreOffice, or similar)

4. System Architecture:

4.1 Workflow:

- **Frame Capture:** RTSP streams are accessed via OpenCV.
- **Detection:** Each frame is processed with YOLOv8 to detect persons (cls_id = 0).
- **Tracking:** Detected persons are assigned unique IDs using distance-based matching.
- **Idle Detection:** If a person remains in the same position for more than 5 seconds, they are considered idle.
- **Logging:** When a new maximum idle count is observed, the data is logged to Excel.
- **Visualization:** Frames are displayed with bounding boxes (green = active, red = idle).

4.2 Flowchart:



5. Implementation:

5.1 Frame Acquisition:

Each camera stream runs on a separate thread. OpenCV continuously fetches frames and stores the most recent frame in memory.

5.2 Detection and Tracking:

- **YOLOv8 model** detects people in the frame.
- Each person is assigned an ID by comparing the current position to previously tracked positions using **Euclidean distance**.

5.3 Idle Detection:

- Movement threshold: **10 pixels**.
- Idle time threshold: **5 seconds**.
- If exceeded, the person is marked **Idle**.

5.4 Excel Logging:

When a new **maximum idle count** is observed:

- Camera name
 - Idle count
 - People count
 - Timestamp
- are appended to idle_report.xlsx.

5.5 Visualization:

Bounding boxes are drawn on frames:

- **Green:** Active persons
- **Red:** Idle persons

6. Results:

6.1 Real-Time Output:

- Live video with bounding boxes and ID labels.
- Accurate differentiation between active and idle individuals.

6.2 Excel Report:

Sample log entry:

7. Limitations:

- Tracking may fail under heavy occlusion.
- Idle detection is sensitive to small camera shakes or noise.
- Currently, only Excel reporting is supported (no database integration).
- Works best with **static camera positions**.

8. Future Enhancements:

Replace simple tracking with **DeepSORT or ByteTrack** for robustness.

- Add **region-based idle detection** (zones of interest).
- Integrate with **databases (MySQL, PostgreSQL)**.
- Generate **alerts (email, SMS, or push notifications)** for idle events.
- Support **multi-camera dashboards**.

9. Conclusion:

This project demonstrates the integration of **YOLOv8, OpenCV, and Excel reporting** for a practical CCTV surveillance system. The system effectively detects and tracks individuals, identifies idle behavior, and generates structured reports for monitoring.

It provides a foundation for more advanced applications in security, workplace monitoring, and behavioral analytics.

Project-2

Automated Packing Arrangement Verification

1. Introduction:

Effective quality control in manufacturing and logistics necessitates dependable identification of packaging configurations. Manual inspections are labor-intensive and susceptible to inaccuracies. This project introduces an automated packing verification system that utilizes computer vision methodologies.

The system evaluates the configuration of brown boxes in a real-time CCTV/RTSP feed against a reference image. Based on the identified positions, it categorizes the packing as:

OK → Arrangement corresponds with reference

NOT OK → Boxes are present but misaligned

EMPTY → No boxes detected

This facilitates ongoing surveillance in packaging facilities, storage areas, and production lines.

2. System Objectives:

1. Detect brown-colored boxes in both reference and live images.
2. Extract box positions using contour detection and geometric filtering.
3. Compare live arrangement with the reference arrangement.
4. Classify the result as OK, NOT OK, or EMPTY.
5. Provide real-time visualization for operators.

3. Tools and Technologies:

- Programming Language: Python 3.11
- Libraries:
 - OpenCV → Image processing, video stream handling
 - NumPy → Matrix operations
- Hardware Requirements:
 - CCTV/Network Camera with RTSP support
 - Standard PC/laptop (GPU not required, runs on CPU)
- Software Requirements:
 - Python environment with OpenCV and NumPy installed

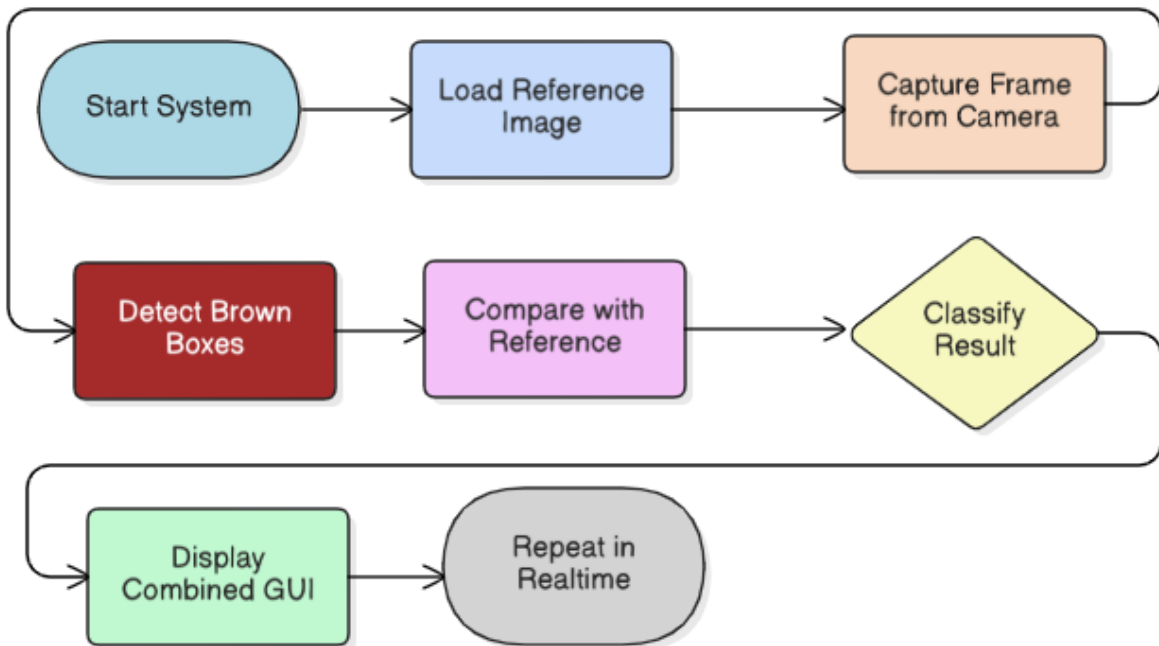
- RTSP-enabled CCTV stream

4. System Architecture:

4.1 Workflow:

1. Load reference image (perfect arrangement Detect boxes in reference and live images using color segmentation).
2. Extract contours and filter based on shape, aspect ratio, and area.
3. Compare positions of detected boxes between reference and live frame.
4. Decision-making:
 - a. No boxes → EMPTY
 - b. Same number and positions within threshold → OK
 - c. Otherwise → NOT OK
5. Display results side-by-side for operator monitoring.

4.2 Flowchart:



5. Implementation:

5.1 Box Detection:

- Convert image from **BGR** → **HSV** color space.
- Define brown color range:

```
lower_brown = np.array([10, 50, 20])
upper_brown = np.array([40, 255, 200])
```

- Apply mask and morphological operations (dilation) to improve detection.
- Extract contours and filter by **area (>2000 px)** and **aspect ratio (0.5–2.0)**.

5.2 Sorting:

Boxes are sorted **top-to-bottom, then left-to-right** to maintain consistent ordering.

5.3 Comparison:

- The center points of reference and test boxes are compared.
- If the distance in **x** and **y** directions is within **threshold = 50**, the arrangement is considered a match.

5.4 Result Classification:

- **EMPTY:** Reference boxes exist but none detected in the test frame.
- **OK:** All boxes match reference positions.
- **NOT OK:** Boxes present but arrangement mismatches.

5.5 Visualization:

- Draw bounding boxes:
 - Blue for reference boxes
 - Red for detected test boxes

6. Results:

6.1 Real-Time Operation:

- The system processes live video feed continuously.

- Comparison performed in each frame ensures immediate feedback.

6.2 Example Outcomes:

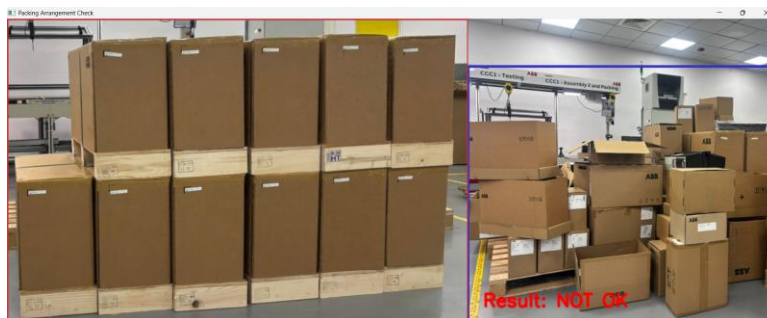
1. Correct Packing:

Result displayed → **OK** (green text).



2. Incorrect Packing (misaligned/missing box):

Result displayed → **NOT OK** (red text).



3. No Boxes Present:

Result displayed → **EMPTY** (red text).



7. Limitations:

- Sensitive to lighting variations (brown detection may fail under different illumination).
- Works best when the camera angle is fixed and consistent with reference image.
- Cannot distinguish between box types; only arrangement is verified.

8. Future Enhancements:

- Use machine learning classifiers for more robust box detection (instead of fixed HSV thresholds).
- Implement 3D pose correction for tilted or rotated boxes.
- Add database or Excel logging for tracking historical results.
- Provide alerts (buzzer, notification, email) when arrangement is NOT OK.
- Support multi-camera feeds for larger packing areas.

9. Conclusion:

This project demonstrates the successful application of OpenCV-based computer vision for automated packing verification. By comparing live camera feeds with a reference image, the system ensures that boxes are arranged correctly in real time.

It reduces manual inspection effort, improves consistency, and provides a scalable solution for industrial quality control.

Project-3

Automated Product Verification

1. Introduction:

In the manufacturing sector, it is essential to verify that products are fully assembled prior to packaging in order to prevent the dispatch of defective items. Conventional manual inspection methods tend to be slow and susceptible to errors. This initiative presents a product verification system based on computer vision, which contrasts a reference image of the finished product with the real-time feed from an RTSP-enabled camera.

The system identifies discrepancies between the two images and assesses whether the product is Complete or Incomplete. This innovative solution has the potential to enhance the automation of quality control procedures in assembly lines, packaging facilities, and warehouses.

2. System Objectives:

1. Capture live frames from an RTSP camera stream.
2. Compare each live frame against a predefined reference image.
3. Detect discrepancies by calculating pixel-level differences.
4. Classify the result as **Complete** or **Incomplete**.
5. Provide real-time visualization of comparison results.

3. Tools and Technologies:

- **Programming Language:** Python 3.11
- **Libraries:**

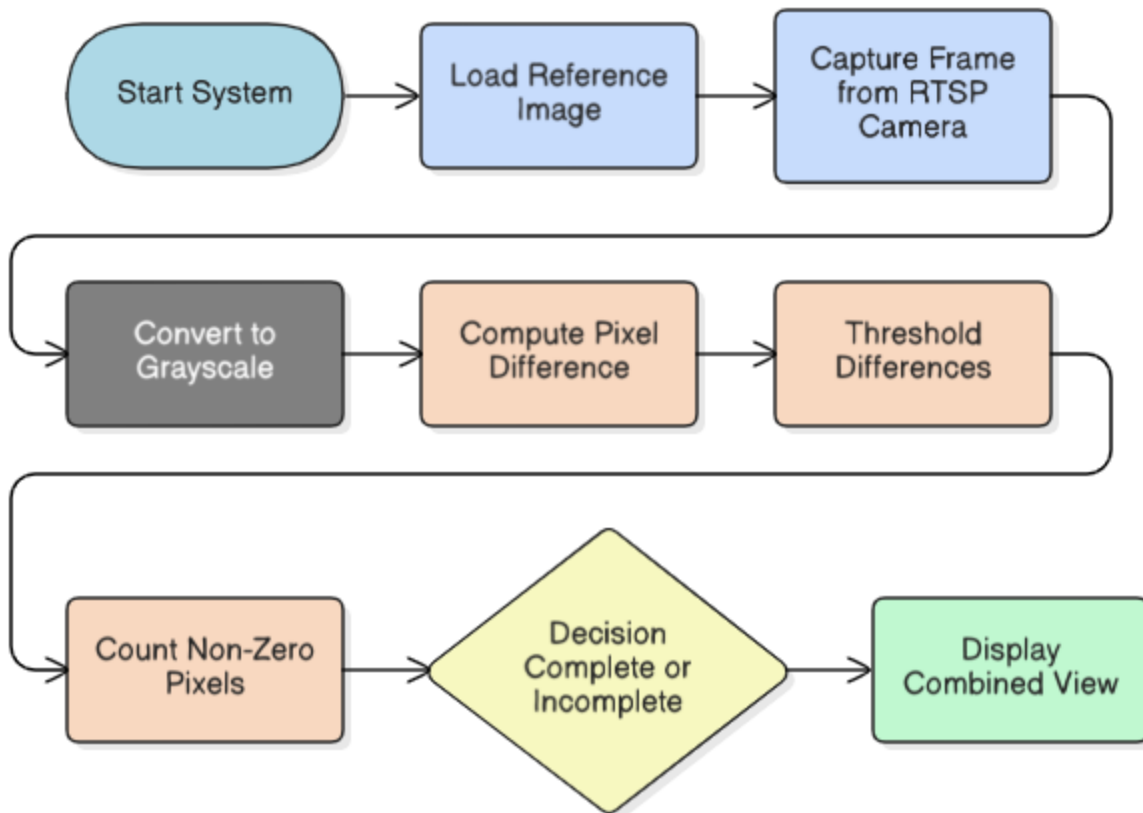
- OpenCV → Image preprocessing, difference calculation, video handling
- NumPy → Array manipulation and mathematical operations
- **Hardware Requirements:**
 - Network/CCTV camera with RTSP support
 - Standard PC/laptop
- **Software Requirements:**
 - Python environment with OpenCV and NumPy installed

4. System Architecture:

4.1 Workflow:

1. Load the reference image (perfect product).
2. Capture frames from the RTSP camera stream.
3. Convert both reference and test images to grayscale for easier comparison.
4. Compute the absolute pixel difference between reference and live frame.
5. Apply thresholding to highlight mismatched regions.
6. Count the number of differing pixels.
7. Decide the result:
 - a. If differences exceed a threshold → Incomplete.
 - b. Otherwise → Complete.
8. Display the reference image, test image, and difference image side-by-side with result text.

4.2 Flowchart:



5. Implementation:

5.1 Image Preprocessing

- Both reference and test images are resized to 400×400 pixels for uniformity.
- Converted from BGR to grayscale to simplify comparison.

5.2 Difference Calculation

- Use `cv2.absdiff()` to compute pixel-wise differences.
- Apply thresholding (`cv2.threshold`) to separate significant differences.

5.3 Decision Criteria:

- Count the number of non-zero pixels in the difference image.
- If `non_zero_count > 5000` → mark as Incomplete ❌.
- Otherwise → mark as Complete ✅.

5.4 Visualization:

- Stack reference image, live test frame, and difference image horizontally.
- Display result text (green for Complete, red for Incomplete).

6. Results:

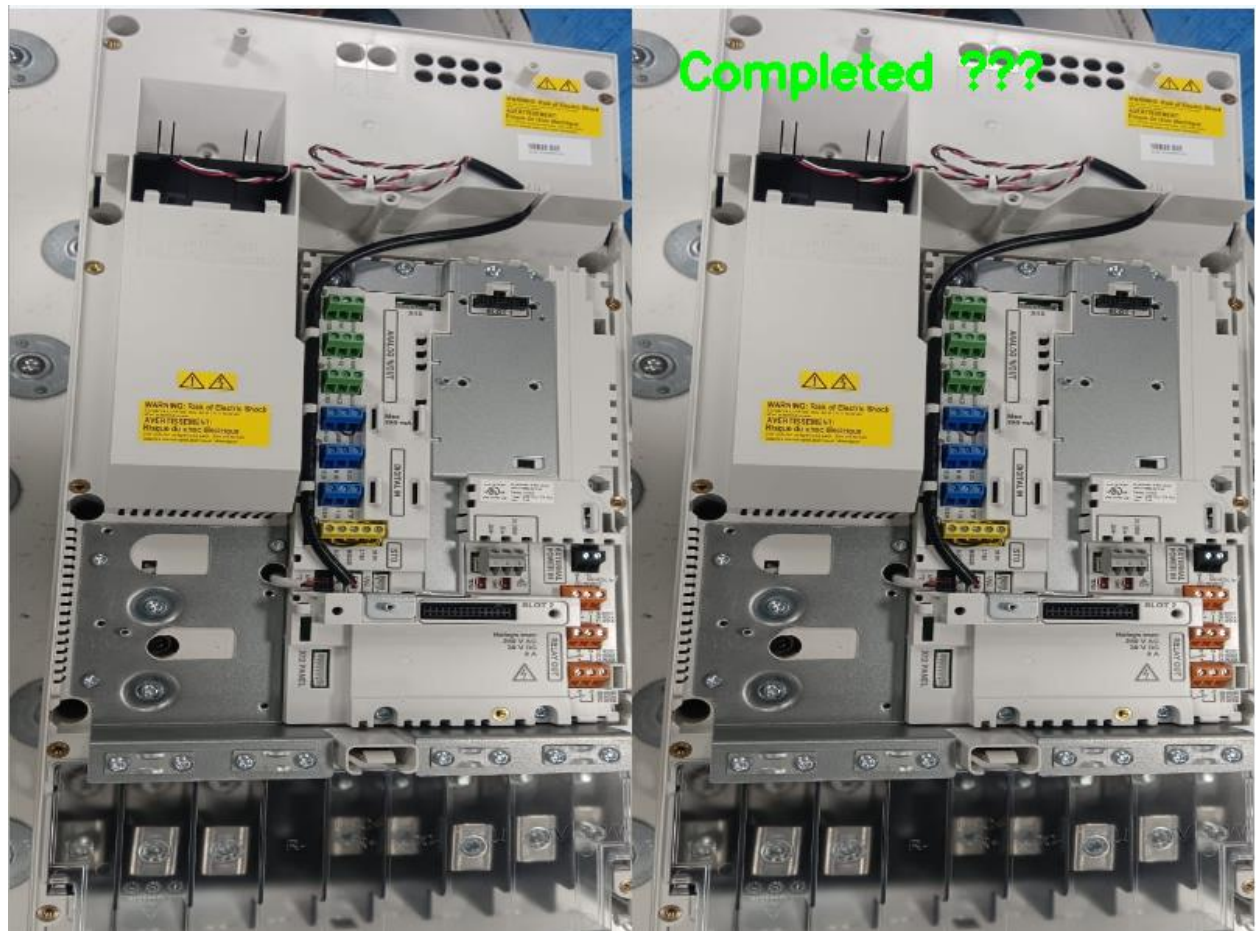
6.1 Real-Time Operation:

- System compares frames continuously in real time.
- Discrepancies are highlighted in the difference image.

6.2 Example Outcomes:

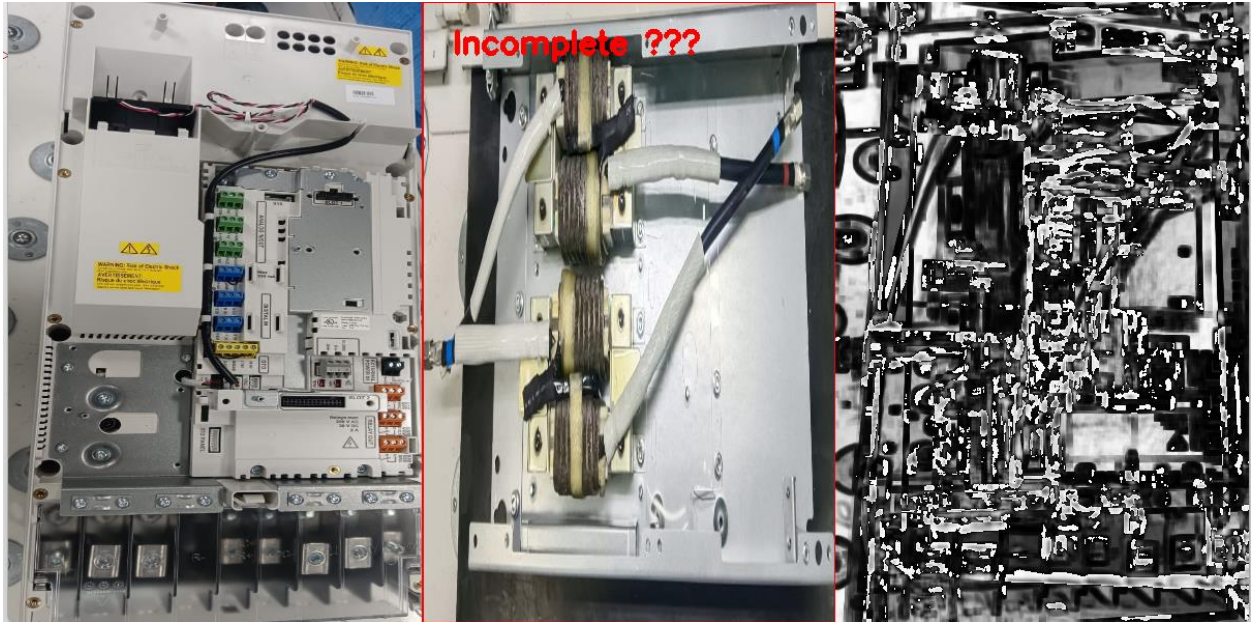
1. Perfect Product:

- Few or no differences detected.
- Output → **Result: Complete** ✓.



2. Missing Component / Wrong Arrangement:

- a. Large differences detected.
- b. Output → **Result: Incomplete** ✖.



7. Limitations:

- Sensitive to camera angle changes — reference and test must be aligned.
- Lighting variations may cause false differences.
- Only checks for visual completeness (not functional correctness).

8. Future Enhancements:

- Use feature-based matching (ORB/SIFT) for more robust comparison.
- Implement image alignment (homography) to handle camera shifts.
- Add automatic logging (Excel/Database) for defect tracking.
- Introduce alerts (buzzer, SMS, email) when incomplete products are detected.
- Train a deep learning classifier for more intelligent defect detection.

9. Conclusion:

This project demonstrates a simple but effective vision-based product verification system. By comparing live camera feeds with a reference image, the system can determine product completeness in real-time.

It provides an efficient solution for automated quality control in production and packaging lines, reducing reliance on manual inspection and improving overall accuracy.

Appendix:

Appendix A: Source Code for Project 1

```
import cv2 from ultralytics import YOLO import threading import time import math from  
openpyxl import Workbook, load_workbook import os
```

```
model = YOLO("yolov8s.pt")
```

```
rtsp_urls = { "Camera 1": "rtsp://admin:Modp09abb@192.535.12.128/Streaming/Channels/701",  
#"Camera 2": "rtsp://username:password@192.168.1.100:554/Streaming/Channels/201" }
```



```

latest_frames = {name: None for name in rtsp_urls}

tracks = {name: {} for name in rtsp_urls} next_id = {name: 0 for name in rtsp_urls}

max_idle = {name: 0 for name in rtsp_urls}

excel_file = "idle_report.xlsx" if not os.path.exists(excel_file): wb = Workbook() ws = wb.active
ws.title = "Idle Report" ws.append(["Camera Name", "Max Idle Count", "People Count",
"Timestamp"]) wb.save(excel_file)

def grab_frames(name, url): global latest_frames cap = cv2.VideoCapture(url,
cv2.CAP_FFMPEG) cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)

if not cap.isOpened():
    print(f"❌ Could not open {name}")
    return

while True:
    ret, frame = cap.read()
    if not ret:
        print(f"⚠️ Failed to grab frame from {name}")
        continue
    latest_frames[name] = frame
    for name, url in rtsp_urls.items(): threading.Thread(target=grab_frames, args=(name, url),
daemon=True).start()

def distance(p1, p2): return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)

def save_to_excel(camera_name, idle_count, people_count): wb = load_workbook(excel_file)
ws = wb["Idle Report"] ws.append([camera_name, idle_count, people_count,
time.strftime("%Y-%m-%d %H:%M:%S")]) wb.save(excel_file)

while True: display_frames = []

for name, frame in latest_frames.items():
    if frame is None:
        continue

    frame_copy = frame.copy()
    results = model(frame_copy)[0]

    people_count = 0
    idle_count = 0

```



```

new_tracks = {}

for box in results.bboxes:
    cls_id = int(box.cls[0])
    if cls_id != 0: # only persons
        continue

    x1, y1, x2, y2 = map(int, box.xyxy[0])
    cx, cy = (x1 + x2) // 2, (y1 + y2) // 2
    people_count += 1

    assigned_id = None
    for pid, pdata in tracks[name].items():
        if distance((cx, cy), pdata["pos"]) < 50:
            assigned_id = pid
            break

    if assigned_id is None:
        assigned_id = next_id[name]
        next_id[name] += 1

    last_pos = tracks[name].get(assigned_id, {"pos": (cx, cy)})["pos"]
    moved = distance((cx, cy), last_pos)

    if moved < 10:
        last_move_time = tracks[name].get(assigned_id, {"last_move":
time.time()}["last_move"])
    else:
        last_move_time = time.time()

    idle_time = time.time() - last_move_time
    is_idle = idle_time > 5

    if is_idle:
        idle_count += 1
        color = (0, 0, 255)
        label = f"ID:{assigned_id} Idle"
    else:
        color = (0, 255, 0)
        label = f"ID:{assigned_id} Active"

```

```

cv2.rectangle(frame_copy, (x1, y1), (x2, y2), color, 2)
cv2.putText(frame_copy, label, (x1, y1 - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)

new_tracks[assigned_id] = {"pos": (cx, cy), "last_move": last_move_time}

tracks[name] = new_tracks

# Update max idle count and save to excel if new max
if idle_count > max_idle[name]:
    max_idle[name] = idle_count
    save_to_excel(name, idle_count, people_count)

# Show counts
cv2.putText(frame_copy, f'{name} | People: {people_count} | Idle: {idle_count} | Max Idle:
{max_idle[name]}',
            (30, 50), cv2.FONT_HERSHEY_SIMPLEX, 1.0, (255, 255, 0), 3)

display_frames.append(frame_copy)

# Display side by side
if len(display_frames) == 2:
    combined = cv2.hconcat(display_frames)
    cv2.imshow("YOLO CCTV Feed (Idle Detection + Excel Logging)", combined)
elif len(display_frames) == 1:
    cv2.imshow("YOLO CCTV Feed (Idle Detection + Excel Logging)", display_frames[0])

if cv2.waitKey(1) & 0xFF == ord('q'):
    break
cv2.destroyAllWindows()

```

Appendix B: Source Code for Project 2

```

import cv2
import numpy as np

def find_brown_boxes(image):
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

    # Define brown color range in HSV
    lower_brown = np.array([10, 50, 20])
    upper_brown = np.array([40, 255, 200])

```

```

mask = cv2.inRange(hsv, lower_brown, upper_brown)

# Morphological operation to fill gaps
kernel = np.ones((5, 5), np.uint8)
dilated_mask = cv2.dilate(mask, kernel, iterations=1)

contours, _ = cv2.findContours(dilated_mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

boxes = []
for cnt in contours:
    x, y, w, h = cv2.boundingRect(cnt)
    aspect_ratio = w / float(h)
    area = cv2.contourArea(cnt)

    if area > 2000 and 0.5 < aspect_ratio < 2.0:
        boxes.append((x, y, w, h))

# Sort boxes top-to-bottom, then left-to-right
boxes.sort(key=lambda b: (b[1], b[0]))
return boxes

def compare_box_positions(ref_boxes, test_boxes, threshold=50):
    if len(ref_boxes) != len(test_boxes):
        return False # Different number of boxes

    for i in range(len(ref_boxes)):
        ref_x, ref_y, ref_w, ref_h = ref_boxes[i]
        test_x, test_y, test_w, test_h = test_boxes[i]

        dist_x = abs((ref_x + ref_w / 2) - (test_x + test_w / 2))
        dist_y = abs((ref_y + ref_h / 2) - (test_y + test_h / 2))

        if dist_x > threshold or dist_y > threshold:
            return False

    return True

def check_arrangement_live(reference_path, rtsp_url):
    ref_img = cv2.imread(reference_path)

```

```

if ref_img is None:
    print("Error: Could not load reference image.")
    return

# Resize reference image
height = 600
ref_img = cv2.resize(ref_img, (int(ref_img.shape[1] * height /
ref_img.shape[0]), height))
ref_boxes = find_brown_boxes(ref_img)

# Open RTSP Stream
cap = cv2.VideoCapture(rtsp_url)
if not cap.isOpened():
    print("Error: Could not open RTSP stream.")
    return

while True:
    ret, test_img = cap.read()
    if not ret:
        print("Error: Failed to grab frame.")
        break

    # Resize frame
    test_img = cv2.resize(test_img, (int(test_img.shape[1] * height /
test_img.shape[0]), height))
    test_draw = test_img.copy()

    test_boxes = find_brown_boxes(test_img)

    # Draw reference and test boxes
    ref_display = ref_img.copy()
    for (x, y, w, h) in ref_boxes:
        cv2.rectangle(ref_display, (x, y), (x + w, y + h), (50, 50,
200), 3)
    for (x, y, w, h) in test_boxes:
        cv2.rectangle(test_draw, (x, y), (x + w, y + h), (200, 50,
50), 3)

    # Determine Result

```

```

if len(ref_boxes) > 0 and len(test_boxes) == 0:
    result_text = "Result: EMPTY"
    result_color = (0, 0, 255) # Red
elif compare_box_positions(ref_boxes, test_boxes):
    result_text = "Result: OK"
    result_color = (0, 255, 0) # Green
else:
    result_text = "Result: NOT OK"
    result_color = (0, 0, 255) # Red

# Put result text
cv2.putText(test_draw, result_text, (30, test_draw.shape[0] - 30),
            cv2.FONT_HERSHEY_SIMPLEX, 1.2, result_color, 3,
cv2.LINE_AA)

# Show combined image
combined = np.hstack((ref_display, test_draw))
cv2.imshow("Packing Arrangement Check", combined)

key = cv2.waitKey(1)
if key == 27: # ESC to exit
    break

cap.release()
cv2.destroyAllWindows()
reference_image = "perfect.jpg" rtsp_url =
"rtsp://username:password@192.168.1.100:554/Streaming/Channels/201" # Replace with your
actual RTSP stream

check_arrangement_live(reference_image, rtsp_url)

```

Appendix C: Source Code for Project 3

```

import cv2
import numpy as np

reference_path = "2.jpg"
ref_img = cv2.imread(reference_path)
if ref_img is None:
    print("Error: Could not load reference image.")
    exit()

ref_img = cv2.resize(ref_img, (400, 400))

```

```
rtsp_url = "rtsp://username:password@192.168.1.100:554/Streaming/Channels/101" cap =  
cv2.VideoCapture(rtsp_url)
```

```
if not cap.isOpened(): print("Error: Could not connect to camera.") exit()
```

```
while True: ret, frame = cap.read() if not ret: print("Failed to grab frame.") break
```

```
# Resize live frame
```

```
test_img = cv2.resize(frame, (400, 400))
```

```
# Convert to grayscale
```

```
gray_ref = cv2.cvtColor(ref_img, cv2.COLOR_BGR2GRAY)
```

```
gray_test = cv2.cvtColor(test_img, cv2.COLOR_BGR2GRAY)
```

```
# Absolute difference
```

```
diff = cv2.absdiff(gray_ref, gray_test)
```

```
# Threshold to highlight differences
```

```
_, thresh = cv2.threshold(diff, 50, 255, cv2.THRESH_BINARY)
```

```
# Count non-zero differences
```

```
non_zero_count = cv2.countNonZero(thresh)
```

```
# Decide result
```

```
if non_zero_count > 5000: # adjust sensitivity
```

```
    result_text = "Result: Incomplete ❌"
```

```
    color = (0, 0, 255) # Red
```

```
else:
```

```
    result_text = "Result: Complete ✅"
```

```
    color = (0, 255, 0) # Green
```

```
# Stack images for display
```

```
diff_bgr = cv2.cvtColor(thresh, cv2.COLOR_GRAY2BGR)
```

```
combined = np.hstack((ref_img, test_img, diff_bgr))
```

```
# Add result text
```

```
cv2.putText(combined, result_text, (50, 50),  
            cv2.FONT_HERSHEY_SIMPLEX, 1.2, color, 3)
```

```
# Show output
```

```
cv2.imshow("Product Verification", combined)
```

```
if cv2.waitKey(1) & 0xFF == ord('q'):  
    break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```